Bullet Time: Safe Reinforcement Learning for Whole-Body Evasion on a Constrained Base

Denis Crismariu
Eindhoven University of Technology
Eindhoven, the Netherlands
d.crismariu@student.tue.nl

Jinyu Zhou
Eindhoven University of Technology
Eindhoven, the Netherlands
j.zhou1@student.tue.nl

Harsh Salkar
Eindhoven University of Technology
Eindhoven, the Netherlands
h.p.salkar@student.tue.nl

Ratanjot Singh
Eindhoven University of Technology
Eindhoven, the Netherlands
r.singh1@student.tue.nl

Tugan Basaran
Eindhoven University of Technology
Eindhoven, the Netherlands
t.basaran@student.tue.nl

Abstract

Humanoid robots operating in unstructured environments must remain balanced while reacting to fast-moving hazards. Prior work treats balance and projectile evasion separately and assumes that the robot can step away from the threat. We study whole-body evasion on a constrained base: the robot stands on a small platform and must dodge flying hazards through whole-body motion while remaining upright. We formulate the problem as a constrained Markov decision process and introduce a modified CRAX environment with explicit fall and bullet-contact costs. To make standing in place on a small support learnable, we modify the CRAX humanoid (flat feet with actuated ankles, a stiffer contact solver, colliding body parts) and keep the bullets from interacting with its dynamics. We train policies with constrained reinforcement learning on a three-level curriculum: (i) one slow bullet, (ii) three bullets at medium speed, and (iii) six fast bullets. We ask two research questions: does the curriculum improve over training directly on the final level (RQ1), and how does the hazard type, small projectiles (bullets) or large obstacles (planes), change what the agent learns and how safe it is (RQ2)?

CCS Concepts

• **Computing methodologies** → **Reinforcement learning**; • **Computer systems organization** → *Robotics*.

Keywords

safe reinforcement learning, constrained Markov decision processes, curriculum learning, humanoid robots

ACM Reference Format:
Denis Crismariu, Jinyu Zhou, Harsh Salkar, Ratanjot Singh, and Tugan Basaran. 2027. Bullet Time: Safe Reinforcement Learning for Whole-Body Evasion on a Constrained Base. In *Proceedings of Proceedings of the 33rd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '27).* ACM, New York, NY, USA, 9 pages.

1 Introduction

Humanoid robots are intended to work in unstructured environments, where hazards can appear with little warnings. Safety in such settings requires more than avoiding collisions, since the robot must also remain upright while it reacts, otherwise, a fall can damage the robot and its surroundings [15, 16]. Reacting quickly to a hazard typically demands whole-body motion, which shifts the center of mass and impedes balance.

Prior learning-based work has largely addressed these two requirements separately. Balance on moving or limited supports has been studied for a quadruped on a six-degree-of-freedom platform [16] and for a biped under tilt and impact perturbations [14]. Humanoid dodging has been demonstrated against thrown balls and against a projectile with proximity-sensing skins in simulation [8]. In both of those works the robot is free to reposition, and one threat is considered at a time.

However, in the real-world settings a robot faces multiple hazards simultaneously, including a narrow support, such as a platform, a ledge or a moving vehicle. We call this a *constrained base*. Evasion and balance then conflict directly, since every avoidance motion moves the center of mass toward the edge of the support, and leaving the support is a safety violation rather than a recoverable error. Reward shaping is hard to control how often such violations occur, whereas the constrained Markov decision process (CMDP) [1] framework allows us to bound the expected cost explicitly [13].

We study this problem with a modified CRAX [13] environment in which a humanoid stands on a round platform and must keep every part of its body out of spherical bullets that fly straight toward it. The bullets of an episode, that is, their number, positions, speeds and aim, form the context of a contextual MDP [3]. The stock CRAX humanoid could not learn to stand still on a small platform, so we modified its feet, contact solver and collision geometry. We train policies with constrained RL on a three-level curriculum: one slow bullet, three medium-speed bullets, and six fast bullets.

2 Related Work

Balance on limited or moving supports. Learning-based stabilization on moving platforms has been shown for legged robots. LASMP [16] trains a quadruped on a 6-DoF moving platform with a self-balancing policy and state estimators. Its stated goal includes avoiding falling off the platform even with limited space. In [14] the authors train a "global self-stabilizer" for a human-sized biped under tilt (±25°) and impact perturbations, and report gains over the model-based controllers that generated its training data. These works show that platform disturbances can be handled by learned policies, but they treat balance as the only task and do not involve a full-body humanoid facing another disturbance.

Dynamic object avoidance. PAC-MAN [15] trains a Unitree G1 to dodge thrown balls from segmentation-masked depth, uses control-barrier-function guidance and an adversarial motion prior during training, and reports 95% success on real throws. Pudasaini et al. [8] train a PPO policy on a Unitree H12 with simulated capacitive and time-of-flight proximity skins against a single projectile, reaching 96–97% avoidance in simulation versus 31% for an MPC baseline.

Safe and constrained reinforcement learning for humanoids. This line of work addresses safety during learning. Constrained-MDP methods optimize return under expected-cost budgets; for example, CRAX [13] provides a fast JAX/MJX benchmark for such methods across difficulty levels. For humanoids, ConstrainedMimic [6] enforces constraints in real time within RL tracking policies, targeting motion tracking rather than reactive behaviour.

Curriculum learning and large-scale simulation. The last line of work concerns how such policies are trained. GPU-parallel MJX training makes large humanoid experiments practical [12, 17], and curriculum learning has been applied to humanoid locomotion [12]. CRAX [13] has also demonstrated that curricula across difficulty levels can improve performance over direct training in harder safe-RL tasks. These curricula vary generic difficulty or task parameters. Authors in [5] studied curricula specifically for constrained RL: their safe curriculum generation approach prefers tasks with few safety violations early on, and improves safety during training and sample efficiency compared with standard curriculum methods and their naively modified safe versions. Such curricula are generated automatically, whereas ours is designed around the structure of the evasion task.

Our contribution. These lines of work treat balance, evasion, safety, and curricula largely in isolation. Platform-balance studies involve no concurrent threat, dodging studies assume a robot that can reposition freely and face one projectile at a time, safety is mostly handled through reward shaping, and curricula are either generic or generated automatically, rather than following the structure of the task. We address these limitations with three contributions. (i) A modified CRAX environment in which a humanoid stands on a bounded platform and dodges bullets, formulated as a contextual MDP whose context is the set of bullets, together with the changes to the CRAX humanoid that make standing in place learnable. (ii) A three-level curriculum that increases the number and speed of the bullets from one slow bullet to six fast ones. (iii) A CMDP formulation with explicit fall and bullet-contact costs, and an empirical study of two research questions: whether the curriculum improves over training directly on the final level, and how the hazard type, small bullets or large planes, changes what the agent learns. Code of trained policies, for both bullets and planes, are available at [4], and project website with videos.

3 Background

Constrained Markov decision processes. We model the task as a constrained Markov decision process (CMDP) [1]. A CMDP is a tuple $(\mathcal{S}, \mathcal{A}, P, \mu, r, \{c_k\}_{k=1}^{K}, \{d_k\}_{k=1}^{K}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(s' \mid s, a)$ the transition kernel, μ the initial-state distribution, and $\gamma \in [0, 1)$ the discount factor. The reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ specifies the task. Each cost function $c_k : \mathcal{S} \times \mathcal{A} \to \mathbb{R}_{\geq 0}$ measures one kind of unsafe behaviour (here, falling and being hit by a bullet), and $d_k \geq 0$ is the budget of cost k.

The agent does not necessarily observe the state. At each step it receives an observation $o_t \in \mathcal{O}$ drawn from an observation function $\Omega(\cdot \mid s_t)$, and it acts according to a stochastic policy $\pi_\theta(a \mid o)$ whose parameters θ are the weights of a neural network. Following π_θ generates a trajectory $\tau = (s_0, o_0, a_0, s_1, \dots)$ with $s_0 \sim \mu$, $o_t \sim \Omega(\cdot \mid s_t)$, $a_t \sim \pi_\theta(\cdot \mid o_t)$ and $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. With $r_t = r(s_t, a_t)$ and $c_{k,t} = c_k(s_t, a_t)$, the expected discounted return and cost returns are

$$J_R(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t \gamma^t r_t\Big], \qquad J_{C_k}(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t \gamma^t c_{k,t}\Big], \quad (1)$$

and the learning problem is

$$\max_\theta \; J_R(\pi_\theta) \quad \text{s.t.} \quad J_{C_k}(\pi_\theta) \leq d_k, \quad k = 1, \dots, K. \quad (2)$$

A policy is *feasible* if it satisfies all K constraints, and the CMDP is feasible if at least one such policy exists. Two properties of Problem (2) matter for our evaluation. First, the constraints bound costs *in expectation* over episodes, so individual episodes may still violate them. Second, the budget determines how much violation is tolerated: if every violation incurs a cost of c_{viol}, a budget d_k allows on average at most d_k/c_{viol} violations per episode (ignoring discounting), so d_k must be chosen relative to the cost of a single violation.

Lagrangian relaxation. A standard way to solve Problem (2) is to convert it into an unconstrained saddle-point problem with Lagrange multipliers $\lambda_k \geq 0$:

$$\min_{\lambda\geq 0} \max_\theta \; \mathcal{L}(\theta, \lambda) = J_R(\pi_\theta) - \sum_k \lambda_k \big(J_{C_k}(\pi_\theta) - d_k\big). \quad (3)$$

The policy parameters are updated by a policy-gradient step on $\mathcal{L}$. The multipliers are updated by a dual step, for example $\lambda_k \leftarrow \max\big(0,\ \lambda_k + \eta\,(J_{C_k} - d_k)\big)$ with step size η, which increases λ_k whenever constraint k is violated ($J_{C_k} > d_k$) and decreases it otherwise, down to zero. Because the primal and dual updates interact, the cost can oscillate around the budget during training, and constraint satisfaction is only approximate [9].

The *PID Lagrangian* [11] damps these oscillations by replacing the dual step with a PID controller on the constraint violation $e_t = J_{C_k} - d_k$:

$$\lambda_t = \text{clip}\left(K_p\, e_t + K_i \sum_{s\leq t} e_s + K_d\,(e_t - e_{t-1}),\ 0,\ \lambda_{\max}\right), \quad (4)$$

where K_p, K_i and K_d are the proportional, integral and derivative gains and $\lambda_{\max}$ caps the multiplier. We update the policy with proximal policy optimization (PPO) [10] on the Lagrangian objective, with separate value networks for the reward and for the cost.

Curriculum learning. Curriculum learning was first proposed for supervised learning [2]. In reinforcement learning, it sequences tasks, or experience, so that an agent can learn a final task that would be too hard to learn from scratch [7]. Curricula differ in what is sequenced, how the sequence is generated, and when the agent moves on [7]. We use a task-level curriculum: an ordered sequence of CMDPs $\mathcal{M}_1, \dots, \mathcal{M}_M$, called *levels*, that share the state and action spaces and differ in the distribution of hazards and in

some parameters of the reward and costs (for example, the number of bullets). Training on level m starts from the policy, value networks and observation normalizer obtained at level $m-1$; the Lagrange multiplier and the optimizer state start again from zero. Our curriculum is hand-designed, and the agent advances after a fixed step budget per level. We evaluate it by comparing final performance and sample efficiency against training directly on $\mathcal{M}_M$ from scratch.

Contextual reinforcement learning. Contextual RL models variation between environment instances through an explicit context, which gives a principled and interpretable way to specify and generate tasks [3]. A contextual MDP is an MDP whose transition, reward and cost depend on a context ξ drawn from a distribution $p(\xi)$ at the start of each episode. Combining it with the CMDP above gives a contextual CMDP, in which every context ξ defines a CMDP $\mathcal{M}_\xi$. We use a context ξ (the bullets of an episode: their number, spawn positions, speeds and aim) to define each episode, and we evaluate policies across the context space. A curriculum level then corresponds to a choice of context distribution $p_m(\xi)$.

4 Methodology

This section defines the bullet environment, in which we train and evaluate policies. Section 4.1 describes the task, the context, the reward and costs, and the curriculum. Section 4.2 describes the plane variant used for RQ2, Section 4.3 the changes made to the CRAX humanoid, and Section 4.4 how the hazards are kept out of its dynamics. The observations, the actions and the use of constrained RL are described with the experimental setup.

4.1 The bullet task

We choose small flying objects as hazards because they arrive with little warning and cannot be handled by a slow, planned motion: a dropped tool, debris, or a thrown ball must be avoided within a fraction of a second, and on a narrow support the robot cannot simply step aside. Its only option is to change its body configuration, which couples avoidance with balance. Methodologically, a spherical bullet is an intentionally simplified threat that (i) is defined by few quantities (position, velocity and radius), so the difficulty can be varied systematically through the number, speed and aim of the bullets, (ii) allows an analytic contact test with every body part, and (iii) yields a dense cost signal, the penetration depth, instead of a sparse contact event. We do not model a specific application. Section 4.2 describes the variant in which the hazard is a large plane, which Section 5.3 uses to study how the size of the hazard changes the problem. Figure 1 shows the setup.

Task. The agent stands on a round platform of radius 0.5 m centered at the origin while up to six bullets fly toward it from the front. It must keep the hitboxes of all body parts out of the bullets and remain upright on the platform for the whole episode. A fall (the torso height outside $[0.6, 2.0]$ m, or the horizontal position of the torso farther than 0.5 m from the platform center) ends the episode. Because stepping off the platform counts as a fall, the agent cannot step away from a bullet; it must avoid it by changing its body configuration while keeping its center of mass over the platform.

Robot and simulation. The robot is a humanoid with 21 actuated joints and 13 body parts, simulated with the Brax generalized backend at a physics time step of 0.003 s, with 5 physics steps per control step (0.015 s); an episode lasts 1000 control steps (15 s). We adjusted the CRAX humanoid and the engine settings so that the robot can stand on the platform (Sections 4.3 and 4.4). The bullets are spheres of radius 0.03 m, up to six per episode. They do not collide with the robot; their contact cost is computed analytically from the collision shapes of the robot.

Context. The context ξ of an episode defines the set of bullets: their number and, for each bullet, its spawn position (x, y, z) and launch velocity. The level fixes the parameters of the distribution $p(\xi)$ (Table 1): the number of bullets, the range of the speed, the range of the spawn position x and the spreads σ_y and σ_z. A bullet is sampled at the start of the episode and again whenever it is respawned. The lateral and vertical positions are $y \sim \mathcal{N}(0, \sigma_y^2)$ and $z \sim \mathcal{N}(0.8, \sigma_z^2)$, clipped to $[-1, 1]$ m and $[0.05, 1.7]$ m, so that most bullets are aimed at the body. The bullet is launched toward $-x$ with speed sampled uniformly from $\mathcal{U}(v_{\min}, v_{\max})$; lateral and vertical velocities are zero. Bullets move at constant velocity (kinematic, no gravity) and are respawned when they pass $x = -1$ m. The environment is therefore a contextual MDP [3], and results can be reported as a function of the context. The policy observes the position and velocity of every bullet relative to the torso, so the sampled context is observed.

Reward and costs. At every step the agent receives

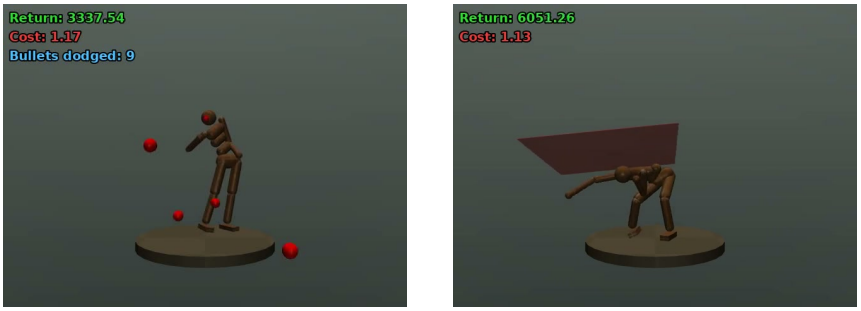
$$r_t = 5 + 5\ \mathrm{clip}\Big(\frac{z_t - 0.6}{1.29 - 0.6},\ 0,\ 1\Big) - 0.1\,\|a_t\|^2, \tag{5}$$

where z_t is the torso height (1.29 m when standing) and a_t the scaled action. The terms reward staying alive, standing tall (which pulls against ducking), and penalize large actions. The bullet cost is

$$c_t = 1.25 \sum_i \sum_j \max\big(0,\ \rho_i + \rho_b - \delta_{ij,t}\big) + c_{\text{fall}}\,\mathbb{I}[\text{fall}_t], \tag{6}$$

where $\delta_{ij,t}$ is the distance between the center of bullet j and the closest point of body part i, ρ_i the radius of part i and $\rho_b = 0.03$ m the bullet radius. Capsules use their axis segment, spheres their center, and the box-shaped feet are treated as spheres with a radius equal to their first half-size. The fall cost is $c_{\text{fall}} = 25$ at every level. Without it, falling on purpose would be a cheap way to escape the bullet cost.

Reward and cost pull in opposite directions. The height term rewards standing tall, while the bullets make every upright posture costly: the lower and more compact the body, the fewer bullets cross it. The robot therefore has to duck or lean only while a bullet passes and stand back up as soon as it has passed. This tension is deliberate, and it is what makes the task suited to constrained RL. A constraint states directly how much cost is tolerated through the budget d, and the Lagrange multiplier adapts the weight of the cost during training until the budget is met. The reward then decides how the robot spends the remaining freedom: it should stay as tall as possible while keeping the cost within budget. Section 5.4 shows where the plane agents end up in this trade-off.



(a) (b)

Figure 1: The two environments, front camera. (a) Bullet environment: the humanoid stands on a round platform of radius 0.5 m, and leaving it counts as a fall; the red spheres are bullets flying toward it. The overlay shows the episode return, the accumulated cost and the number of bullets dodged so far. (b) Plane environment, level 3: a tilted 1.5 m square plane slides through the platform (radius 0.75 m) and the humanoid has to duck under it.

Curriculum. The levels are (i) one slow bullet, (ii) three medium-speed bullets and (iii) six fast, more tightly aimed bullets (Table 1). Each level starts from the policy, value networks and observation normalizer obtained at the previous level, and is trained for a fixed number of environment steps. The reward and costs are the same at every level.

4.2 The plane task

For RQ2 we use a second environment, in which the hazard is a single large obstacle instead of several small projectiles. We model the obstacle as a tilted square plane and call this the plane environment; the first, in which the projectiles are modeled as spherical bullets, is the bullet environment. Both environments share the humanoid (Section 4.3), the simulation and episode length, the base reward of Equation (5), the fall height and the cost weight of 1.25 per metre of penetration. Table 2 lists how they differ, and Table 1 gives the levels of both. The plane is not part of the physics model: the environment moves it at every step (Section 4.4).

Cost. The agent must keep the hitboxes of all body parts out of the plane, which is tilted randomly each time it appears. The cost is

$$c_t = 1.25 \sum_i \max\left(0,\ d_{i,t}\right) + c_{\text{fall}}\, \mathbb{I}[\text{fall}_t], \tag{7}$$

where $d_{i,t}$ is the penetration depth of the hitbox of part i into the plane, in metres. The fall cost of 100 on levels 2 and 3 is more than the cost of standing under every plane, so falling on purpose never pays. Kneeling counts as a fall on level 3, since otherwise a policy can kneel for the whole episode, which clears every plane (Section 5.4).

Reward. The two extra terms of Table 2 shape how the robot ducks: the upright reward and the cheap height reward at 1.0 m make ducking with the knees cheaper than bending at the waist, and the stillness penalty discourages lunging across the platform.

4.3 Changes to the CRAX humanoid

CRAX inherits its humanoid from Brax, where it is used for locomotion tasks that reward forward progress. Standing in one place on a small support for a whole episode, while the body reconfigures quickly, stresses the model in ways that walking does not. We made the following changes, each motivated by a failure we observed.

Flat feet with actuated ankles. The Brax humanoid has spherical feet rigidly attached to the shins. Rolling contact is sufficient for walking, where the robot is always moving, but it gives no support polygon for standing still: with sphere feet the robot did not learn to stand on the platform within $3 \cdot 10^7$ steps, with or without additional upright and stillness rewards. We replaced them with the flat box feet (26×10 cm) of the `dm_control` humanoid and added the joints the standing posture needs: two ankle joints per foot (pitch and roll), so the feet can stay flat on the floor while the legs and torso move above them. A flat foot only helps if the ankle can hold it in place against the weight of the body, so each new joint is driven by a strengthened motor (gear 50) and has passive stiffness and damping that pull it back toward the neutral position. The robot therefore has 21 actuators instead of 17. With flat feet and actuated ankles it learned to stand for the full episode.

Stiffer contact. Box feet touch the floor at their corners, and with Brax's 8 solver iterations these contacts did not converge: the feet sank up to 54 mm into the floor. We raised the solver iterations to 32 and halved the contact time constant to 0.01 s.

Colliding body parts. In the Brax humanoid only the feet collide with the floor, so a kneeling robot passes through it. We made the head, torso, waist, thighs, shins and forearms collide as well, so the robot can kneel or sit physically and the environment can detect a body part resting on the floor.

Platform and reward. The Brax backend cannot collide cylinders, so the round platform is visual only, and a hidden infinite floor provides the contacts. The platform edge is enforced by the environment as a fall rule. Brax's humanoid reward pays for forward velocity; we replace it with terms for staying alive and standing tall (Equation 5). While learning to stand we also used a reward for keeping the torso vertical and a penalty on the horizontal velocity of the center of mass, which were needed for standing to be learned in our runs.

Table 1: Difficulty levels of the two environments. Bullets: speed, spawn distance x and spreads σ_y, σ_z of the context distribution (Section 4.1). Planes: heights are those of the center of the plane.

Level	Hazard	Targeted behaviour
Bullet environment		
1	1 bullet, speed $\mathcal{U}(0.75, 1.5)$ m/s, $x \in [3.0, 5.0]$ m, $\sigma_y, \sigma_z = 0.30, 0.40$ m	Stay balanced while a single slow bullet passes
2	3 bullets, speed $\mathcal{U}(1.5, 3.0)$ m/s, $x \in [2.5, 5.0]$ m, $\sigma_y, \sigma_z = 0.22, 0.35$ m	Dodge several medium-speed bullets
3	6 bullets, speed $\mathcal{U}(3.0, 5.0)$ m/s, $x \in [2.0, 5.0]$ m, $\sigma_y, \sigma_z = 0.18, 0.30$ m	Dodge many fast, tightly aimed bullets and recover to upright
Plane environment		
1	no plane cost, no fall cost; a hidden plane moves as at levels 2 and 3, so that the observation normalizer has seen its ranges	Stand upright on the platform
2	every 3.75 s a plane at a height of 0.9–1.9 m, tilt up to 20°, offset up to 0.3 m, comes down from above at 0.75 m/s and stops; a quarter of the planes slide through from 3 m away at 1.5 m/s instead	Duck or lean under a plane at a random height
3	1.5 m square planes slide through at 1.5 m/s from 2.0 m away in random directions, about one every 2.7 s, tilt up to 20°, height 0.9–1.7 m	Dodge planes arriving repeatedly and recover to upright

Table 2: How the two environments differ. Both use a 1000-step episode, 2048 parallel environments, batch size 1024, 32 minibatches, unroll length 10, learning rate $3 \cdot 10^{-4}$, discount 0.97 and about 100M environment steps per agent.

	Bullets	Planes
Hazard	up to 6 spheres flying straight from the front	one tilted square plane, 1.5 m wide at level 3
Avoided by	moving a single body part or leaning	crouching or moving the whole body
Hazard cost	penetration depth, Eq. (6)	penetration depth, Eq. (7)
Platform radius	0.5 m	0.75 m
Fall	torso height outside $[0.6, 2.0]$ m, or torso more than 0.5 m from the center	torso height outside $[0.6, 2.0]$ m, or a body part on the floor past the edge; level 3: any non-foot body part on the floor
Fall cost	25 at every level	0 / 100 / 100
Extra reward	none	upright torso (5 / 3 / 3), stillness penalty; levels 2–3: a torso at 1.0 m keeps 90% of the height reward
Hazard in level-1 observation	only the first bullet slot	yes, hidden and without cost
Algorithm	PPO-Lag, budget 0.1, no cap	PID Lagrangian, limit 5, cap 30
Entropy, updates per batch	10^{-2}, 4	10^{-3}, 8

4.4 Hazards that do not act on the robot

The hazards never collide with the robot, and their cost is computed analytically from its collision shapes. Keeping them in the physics model as bodies moved by joints still changed the robot's dynamics, because the Brax generalized backend inverts the mass matrix only approximately (a property of our environments, not of MuJoCo in general). The extra joints stretched the range of the mass matrix from about 0.03–42 to 0.002–1000, and a policy that stands for a full episode in Brax then fell after 59–98 steps. We therefore either make the bullets very light (1 g) and invert the mass matrix exactly, which is correct but slower, or remove the hazards from the physics model altogether: they are plain arrays advanced by the environment at every control step and drawn only in renders. The second option keeps Brax's approximate inverse, adds no degrees of freedom, and lets the number of hazards change freely, the plane environment (Section 4.2) is built in this way.

5 Experiments

The experiments answer the two research questions stated in Section 2. Section 5.1 describes the setup that both share. Sections 5.2 and 5.3 each state a question and answer it directly, and Section 5.5 discusses why a curriculum may or may not help.

5.1 Experimental setup

Agents. We compare two agents, both trained with PPO-Lag. The *Curriculum* agent is trained on levels 1, 2 and 3 in turn, with an equal share of the step budget per level and the warm start of Section 3. The *Direct* agent uses the same algorithm, networks, settings and total number of environment steps, but is trained only on the final level, level 3 (six fast bullets). The two agents therefore differ in exactly one factor, the curriculum.

Constrained RL for this task. Bullet and fall costs are summed into a single cost with one constraint, whose budget is $d = 0.1$ per episode (the Lagrange multiplier is updated from the average cost per step against $d/1000$). A single fall (cost 25) therefore already violates it, and the budget allows at most about one fall in 250 episodes. The agents use the Lagrangian method of PPO-Lag [9]: after every training iteration the multiplier is increased by 10 times the amount by which the average cost per step exceeds $d/1000$, and kept non-negative, with no upper bound.

Observations and actions. The observation has 328 values at every level and for any number of bullets, so that the networks transfer between levels: the height and orientation of the torso and the 21 joint angles (26 values), the corresponding velocities (27), the inertia and mass (130) and the center-of-mass velocity (78) of each of the 13 body parts of the robot, the actuator forces (27), the relative position and velocity of each bullet, zero-padded to six bullets (36), and the position of the torso on the platform with the platform radius and the distance to its edge (4). The bullets enter the observation only through these 36 values; their own bodies are not part of the body-part entries. Observations are standardized by a running mean and standard deviation. The action has 21 dimensions in $[-1, 1]$, one per actuator, and is scaled to the actuator range $[-0.4, 0.4]$.

Table 3: Last evaluation of each level (128 episodes, one run). Steps are the environment steps trained on that level; falls are the share of episodes that end in a fall (fall cost / 25); λ is the Lagrange multiplier at that point. Cost and return are mean ± standard deviation over episodes.

Agent, level	Steps	Cost	Bullet cost	Falls	Return	λ
Curriculum, 1	34.1M	2.15 ± 6.05	0.59	6.3%	11381 ± 780	73.2
Curriculum, 2	34.1M	3.11 ± 4.82	2.13	3.9%	11460 ± 989	47.8
Curriculum, 3	34.1M	3.84 ± 5.18	2.67	4.7%	10997 ± 1403	71.8
Direct, 3	100.9M	6.49 ± 4.66	5.51	3.9%	11261 ± 943	92.2

Training. We use 2048 parallel environments, batch size 1024, 32 minibatches, unroll length 10, 4 updates per batch, learning rate $3 \cdot 10^{-4}$, entropy coefficient 10^{-2} and discount 0.97. The policy is an MLP with four hidden layers of 32 units; the reward and cost value networks have five hidden layers of 256 units. Both agents use 100M environment steps in total, so the Curriculum agent spends about a third of them on each level. Every agent is evaluated on 128 episodes before training and after each quarter of the steps of the level it is trained on.

Metrics. The reward and the costs are computed in the same way at every level, but their values are not comparable between levels: the number of bullets changes the scale of the bullet cost, and a fall ends the episode and with it the return. We therefore use the *fall rate*, the share of evaluation episodes that end in a fall (the fall cost divided by 25), as the main measure of safety, since it means the same at every level and for both agents. We report the cost, split into bullet and fall cost, and the return as secondary measures. With 128 episodes, the standard error of a fall rate of about 4% is about 1.8 percentage points. Each agent was trained once.

5.2 RQ1: is the curriculum better than learning directly on the final level?

Figure 2 (bottom) shows the training curves and Table 3 the last evaluation of each level. The direct agent falls in fewer than 10% of the episodes from its first evaluation (25M steps) on, while the curriculum agent falls in every episode again at the start of levels 2 and 3 (Section 5.5) and needs about 17M steps on each to get back below 10%. At the end, the two agents fall equally often (6 and 5 of 128 episodes, within the standard error) and reach the same return. Neither meets the budget, which allows a fall in about 0.4% of the episodes, so the multiplier stays high. The difference is in the bullet cost, which is about half as large for the curriculum agent: it is safer with respect to bullet contact, but not with respect to falls. Counted in total steps, the direct agent is much faster (at 76M steps its cost is 6.1, while the curriculum agent, after 67M steps on easier levels, is at 27); counted in steps on the final level, the two are comparable (a cost of about 6 after 25M and 17M steps).

The curriculum is therefore not necessary to learn this task, it does not reduce the falls, the return or the number of steps compared with direct training, and its only measurable benefit is a lower bullet cost. This is the result of one run per agent, so even the bullet-cost gap is indicative, thus, further seeds are needed to confirm it.

Table 4: Curriculum versus direct training with the plane-shaped hazard, on level 3 (one seed). Mean of the last 5 evaluations; the last column is the evaluation closest to 30M steps on level 3, which compares equal experience on that level.

Agent	Cost	Hazard cost	Falls	Return	Height / upright	Cost at 30M
Curriculum (levels 1–3)	52.0	40.7	11.2%	12402	4782 / 2866	41.1
Direct (level 3)	10.3	7.5	2.8%	10136	3981 / 1295	25.5

Table 5: Direct training on plane level 3 while the task was developed (mean of the last 5 evaluations, about 100M steps each, one seed). The last row is the Direct agent of Table 4.

Setting	Cost	Hazard	Falls	Return	Behaviour
Limit 25, cap 10, planes 1.0–1.7 m	20.7	19.3	1.4%	12167	within budget, never ducks
Limit 5, cap 10	7.4	5.4	2.0%	11387	shallow knee bend
Limit 5, cap 30, planes 0.9–1.7 m	3.5	2.7	0.8%	9328	kneels all episode
+ kneeling is a fall (final)	10.3	7.5	2.8%	10136	crouches and ducks

5.3 RQ2: how do small projectiles and large obstacles differ in how an agent learns to avoid them and how safe it becomes?

Bullets are small, they can be avoided by moving a single body part or by leaning. A plane is large: it covers much of the body, and the robot can only avoid it by crouching or by moving its whole body. We ask how the hazard type, small projectiles (bullets) or large obstacles (planes), changes what the agent learns and how safe it is, for both the Direct and the Curriculum agent, using the fall rate as the main measure.

Plane experiment. The plane agents use the settings of Table 2 with seed 69. The curriculum gave 20M, 50M and 30M steps to levels 1–3, and the direct agent all 100M to level 3 (Table 4). Both use the final level-3 setup, in which a non-foot body part touching the floor is a fall; Figure 2 shows their training curves, Figure 3 where they end up in the trade-off between reward and cost, and Table 5 the direct runs that led to this setup.

The two hazards behave differently under the curriculum. With bullets, the curriculum and the direct agent end with the same fall rate. With planes, the curriculum was clearly worse: at the end it had five times the cost and four times the falls of the direct agent, and it was also behind at the same number of steps on level 3 (Table 4, Figure 2). Neither agent reached the limit of 5. The cause was level 2, whose limit was out of reach (Section 5.5). The direct agent stood after 17M steps and then lowered its hazard cost steadily by crouching and leaning, at the expense of height and upright reward (Section 5.4). The curriculum result repeats an independent run with the same settings (final return and cost 12777 and 52.0 there, 12707 and 44.6 here). A large hazard thus pushes the robot toward the lowest postures, close to the fall height, where the avoidance of the hazard and the fall rule conflict, and the budget of every level has to be chosen with that in mind, that is small bullets leave more ways to avoid them, which may be why the curriculum did not suffer the same failure with bullets, but we have not tested this.

These two experiments are not a controlled comparison of the hazards, because they differ in more than the hazard (Table 2): the

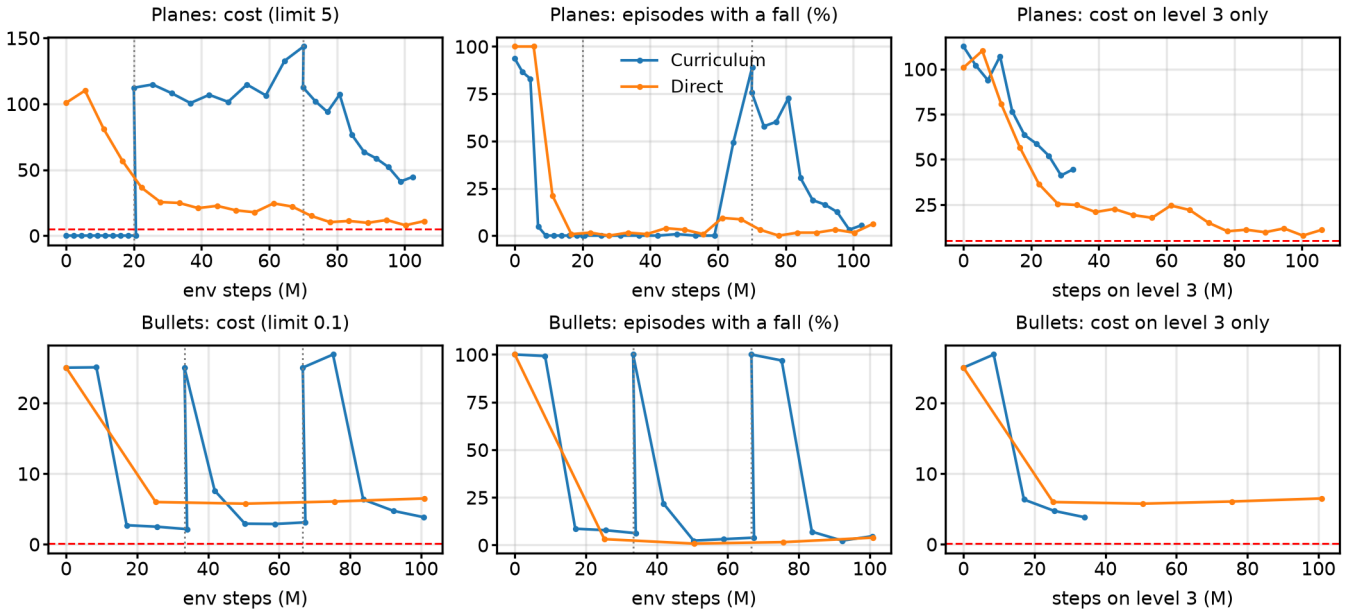


Figure 2: Curriculum (blue) versus Direct (orange) training, planes (top) and bullets (bottom); each row has its own scale (Table 2). Left and middle: cost and falls against total environment steps, with dotted lines at the curriculum's level changes. Right: cost against the steps spent on the last level. Red dashed lines: the cost limit. Plane level 1 has no fall cost, so until the agent stands its falls are a lower bound from the time it survived.

platform, the fall rule and fall cost, the reward terms, the algorithm and its settings. The costs of the two hazards cannot be compared, and the fall rates only qualitatively.

5.4 Standing tall versus avoiding the hazard

Both environments set a reward for standing tall against a cost for being hit (Equations (5)–(7)). The tension is intended: without the height reward, the safest policy would be to get as low as possible and stay there for the whole episode, which avoids the hazard without dodging it. The task is meant to make the agent duck only while a hazard passes and stand back up afterwards. Most of our design work on the plane environment went into keeping this static low posture from being the answer, and the plane runs show how the agent moves between the two ends of the trade-off.

The budget sets the posture. The direct runs on plane level 3 (Table 5) show the trade-off moving with the budget and the multiplier cap. With a budget of 25 the constraint was easy to meet, so as the multiplier came down the policy spent the slack on standing taller: its hazard cost rose from 13 to 20 while its height reward rose from 3340 to 4440, and it never ducked under the low planes. With a budget of 5 it ducked, but only with a slight bend of the knees, which cleared the planes it was trained on (1.0–1.7 m) but not planes at 0.9–1.2 m, so we lowered the range to 0.9–1.7 m. With the multiplier cap raised from 10 to 30, the cost finally outweighed the height reward, and the policy took the static answer: it knelt at the start of the episode and stayed there, giving up more than half of the height reward (about 2000 of 5000) for a hazard cost of 2.7. Kneeling was not yet a fall, and it clears every plane.

Closing the static answers. Each static answer was closed with a rule rather than by retuning the cost. The torso must stay above 0.6 m, so lying down is a fall, and a fall costs 100, more than standing under every plane of an episode, so falling on purpose never pays. Kneeling was made a fall by counting any non-foot body part on the floor. An early level-3 policy ducked by bending at the waist, which lowers the head while keeping the torso, and with it the height reward, high. The upright reward and a height reward that already pays 90% at a torso height of 1.0 m make ducking with the knees cheaper than bending.

Where the final agents ended. The two plane agents ended at opposite ends of the trade-off (Figure 3). The curriculum agent stands tall and straight and accepts the hazard (Table 4). The direct agent first cut its plane cost from 56 to 17 by tilting its torso while keeping its height, and only then crouched lower (Figure 4); its final policy holds the torso at 0.92 m on average (1.29 m standing) without falling. It still ducks when a plane comes, dropping its head by 0.32 m within 0.25 s, since a held crouch does not clear the lowest planes (0.9 m). The cheap duck to 1.0 m, introduced against bending at the waist, made a moderate crouch almost free, so the static answer moved from kneeling to crouching rather than disappearing. A steeper height reward below standing would push it back up, at the risk of the curriculum agent's answer of not ducking at all.

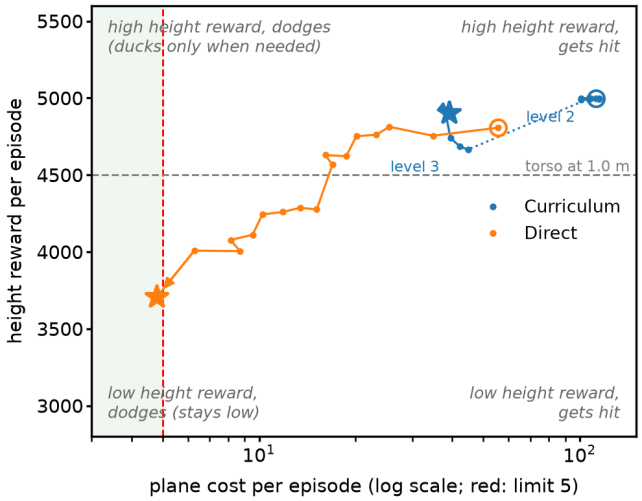


Figure 3: Trade-off between standing tall and avoiding the plane. Each point is one evaluation, from the first (circle) to the last (star); the dotted line marks the curriculum's switch from level 2 to level 3. The red line is the cost limit (the cost axis is logarithmic), and the grey line is the height reward of a torso held at 1.0 m for the whole episode. Evaluations with more than 20% falls are left out, since falls lower every reward. The goal is the top left: stand tall and duck only while a plane passes.

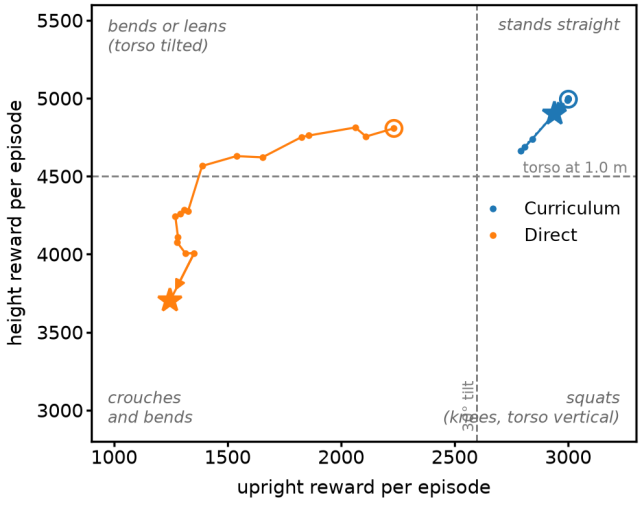


Figure 4: How the plane agents duck, shown with the same evaluations as Figure 3. A lower height reward means a lower torso (squatting); a lower upright reward means a more tilted torso (bending or leaning). The vertical grey line is a torso tilted by 30° for the whole episode. The direct agent first tilts its torso and then crouches; the curriculum agent stays upright.

5.5 Why a curriculum may or may not help

Direct training is never worse. In both environments, training directly on the last level gave at least as few falls as the curriculum (Sections 5.2 and 5.3), and with planes clearly fewer. The only measured benefit of the curriculum was the lower bullet cost.

Why the curriculum fails here. Both environments show the same weakness: the skill does not carry over between levels. With bullets, every episode of the first evaluation on levels 2 and 3, before any training on them, ended in a fall within about 30 steps, as for an untrained policy, so each level had to learn standing again before it could improve on the bullets. A likely cause is the observation normalizer: on level 1 only the first bullet slot is ever non-zero, and on level 2 only the first three, so the slots of the added bullets have zero variance. The normalizer divides by a standard deviation clipped below at 10^{-6} and does not clip its output, so the first bullet in a new slot, 2–5 m away, becomes an input of the order of 10^6 to weights that never saw it. With planes, the hazard was already in the observation from level 1 on, so the cause is the behaviour learned on level 2. For 40M steps the policy stood tall and took every plane (cost 100–115, no falls, full height and upright reward), although the multiplier was at its cap from a local optimum in which the agent stands tall and accepts the cost, because the budget of the level was out of reach and the capped multiplier could not make the cost outweigh the height reward. Level 3 had to unlearn it, and its multiplier restarted from zero and was back at the cap only after about 11M of its 30M steps; it recovered by standing up again rather than by ducking. The direct agent never learns this posture: it meets the hazard from its first episodes, falls in every episode for the first 6M steps, and then learns to stand and to avoid the hazard together.

When could a curriculum help? A curriculum is meant for a final task that is too hard to learn from scratch [7]. Our final levels are not: in both environments the direct agent stood on the last level, with the hazard present, within its first 25M steps, and its cost kept falling from there. To make the bullet task tractable we simplified it in several ways: the bullets all come from the front, they never push the robot, and their cost is computed analytically from the collision shapes of the body. Most of our design effort also went into the final levels; the earlier levels were derived from them by reducing the number, speed or height of the hazards, and were not designed so that each level teaches the skill the next one needs. In the plane curriculum the skill that level 2 taught, standing tall, even works against the final one. Safe curricula keep the early tasks low in violations [5]; ours did the opposite, since the intermediate plane level had a budget the agent could not meet, which kept the multiplier at its cap and taught it to accept the cost. Our results suggest three conditions for a curriculum to help in this kind of task: the observations should have the same statistics on every level (for example, the normalizer should see all bullet slots from level 1 onward), each level should require the avoidance behaviour of the next rather than hazards that can be ignored, and the budget of every level should be reachable, so that the policy is not pushed into a posture it later has to unlearn. A possible explanation for the one measured benefit of the curriculum, the lower bullet cost, is that learning on fewer and slower bullets first shapes the avoidance

of bullets; we have not tested this, and with one run per agent it cannot be separated from the effect of the seed.

6 Conclusion and Future Scope

We studied whole-body evasion on a constrained base with a constrained-RL environment in which a humanoid dodges bullets on a small platform. In a comparison of one run per agent, a curriculum of one, three and six bullets did not reduce the fall rate, the return or the number of steps compared with direct training on the final level, but halved the cost of the bullets that reach the robot. With planes, training directly on the last level was clearly better: the curriculum ended with five times the cost and four times the falls, because level 2, whose budget was out of reach, taught it to stand tall and accept the hazard, a habit the last level could not undo. In both environments the skill did not carry over cleanly between levels, which limits what a curriculum can give here. Large hazards push the robot toward low postures where evasion and the fall rule conflict, so the height reward, the fall rules and the budget have to be set together, and a curriculum needs a reachable budget at every level.

Future scope. Experiments. Our results come from one run per agent; more seeds are needed to confirm them, in particular the lower bullet cost of the curriculum. A second curriculum should fix the problems we found, with a normalizer that sees every bullet slot from level 1 onward and a reachable budget at every level, and could be compared with automatically generated safe curricula [5]. Small and large hazards should also be compared under the same platform, fall rule, reward and algorithm, which our two environments do not share.

Harder and more varied hazards. All bullets fly straight from the front at a constant velocity. Bullets from the sides and from behind, from above and below, along curved (ballistic) paths and with varying sizes would force the robot to use more of its body and to turn toward a threat. Hazards that physically push the robot when they hit, instead of only adding cost, would couple evasion and balance more strongly, as would a platform that moves or tilts [16].

More dynamic evasion. Our best plane policy avoids most of the cost by holding a crouch for the whole episode and ducking a little further when a plane comes. The intended behaviour is to stay upright and move only while a hazard passes. A height reward that drops more steeply below standing, a cost on time spent crouched, or an evaluation that measures how quickly the robot returns to upright after each hazard would push the policy toward short, dynamic dodges, and toward combining ducking, leaning and turning rather than one fixed posture.

Perception. The policy observes the exact position and velocity of every bullet, in a fixed buffer of six slots that is zero-padded when fewer bullets are present. This caps the number of hazards, makes the input depend on the order of the slots, and caused the normalizer problem between levels. A set-based encoder, such as attention over a variable number of objects, would remove the fixed buffer, and learning from camera or depth images, as PAC-MAN does for thrown balls [15], would let the robot detect any number of objects itself, under the noise and occlusion of real sensors, and is a step toward a policy that runs on a real humanoid.

References

[1] E. Altman. 1999. *Constrained Markov Decision Processes.* Chapman and Hall.

[2] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML '09)*. 41–48. doi:10.1145/1553374.1553380

[3] Carolin Benjamins, Theresa Eimer, Frederik Schubert, Aditya Mohan, Sebastian Döhler, André Biedenkapp, Bodo Rosenhahn, Frank Hutter, and Marius Lindauer. 2023. Contextualize Me – The Case for Context in Reinforcement Learning. arXiv:2202.04500 [cs.LG] https://arxiv.org/abs/2202.04500

[4] Denis Crismariu, Jinyu Zhou, Harsh Salkar, Ratanjot Singh, and Tugan Basaran. 2026. Bullet Time: code and videos. https://github.com/crismariudenis/CRAX-Denis-Fork.git. Project page: https://sratanjot1.github.io/Research-Topics-/.

[5] Cevahir Koprulu, Thiago Simão, Nils Jansen, and ufuk topcu. 2025. Safety-Prioritizing Curricula for Constrained Reinforcement Learning. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. 6138–6166. https://proceedings.iclr.cc/paper_files/paper/2025/file/120ed726cf129dbeb8375b6f8a0686f8-Paper-Conference.pdf

[6] Daniel Morton, Pranit Mohnot, and Marco Pavone. 2026. Constrained Whole-Body Tracking for Humanoid Robots. arXiv:2606.00374 [cs.RO] https://arxiv.org/abs/2606.00374

[7] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. 2020. Curriculum learning for reinforcement learning domains: a framework and survey. *J. Mach. Learn. Res.* 21, 1, Article 181 (Jan. 2020), 50 pages.

[8] William Xie Niraj Pudasaini, Carson Kohlbrenner. 2026. Whole-Body Dynamic Obstacle Avoidance with Humanoid Skin. (2026).

[9] Alex Ray, Joshua Achiam, and Dario Amodei. 2019. *Benchmarking Safe Exploration in Deep Reinforcement Learning.* Technical Report. OpenAI.

[10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] https://arxiv.org/abs/1707.06347

[11] Adam Stooke, Joshua Achiam, and Pieter Abbeel. 2020. Responsive safety in reinforcement learning by PID lagrangian methods. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*. Article 847, 11 pages.

[12] Yusuke Tanaka, Alvin Zhu, Quanyou Wang, and Dennis Hong. 2025. Mechanical Intelligence-Aware Curriculum Reinforcement Learning for Humanoids with Parallel Actuation. In *2025 IEEE-RAS 24th International Conference on Humanoid Robots (Humanoids)*. 882–889. doi:10.1109/Humanoids65713.2025.11203130

[13] Tristan Tomilin, Mourad Boustani, Mickey Beurskens, Khoi H. B. Nguyen, and Thiago D. Simão. 2026. CRAX: Fast Safe Reinforcement Learning Benchmarking. arXiv:2606.20376 [cs.LG] https://arxiv.org/abs/2606.20376

[14] Weiguo Wu, Liyang Gao, and Xiao Zhang. 2022. A Stability Training Method of Legged Robots Based on Training Platforms and Reinforcement Learning with Its Simulation and Experiment. *Micromachines* 13, 9 (2022). doi:10.3390/mi13091436

[15] Lizhi Yang, Junheng Li, and Aaron D. Ames. 2026. PAC-MAN: Perception-Aware CBF-RL for Whole-Body Safety in Humanoid Dodgeball. *arXiv preprint arXiv:2607.28623* (2026), 1–10.

[16] Minsung Yoon, Heechan Shin, Jeil Jeong, and {Sung Eui} Yoon. 2024. Learning-based Adaptive Control of Quadruped Robots for Active Stabilization on Moving Platforms. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2024 (IEEE International Conference on Intelligent Robots and Systems)*. 701–708. doi:10.1109/IROS58592.2024.10802150

[17] Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder A. Kahrs, Carmelo Sferrazza, Yuval Tassa, and Pieter Abbeel. 2025. MuJoCo Playground. arXiv:2502.08844 [cs.RO] https://arxiv.org/abs/2502.08844